Università degli Studi di Firenze

Dipartimento di Ingegneria dell'Informazione

SISTEMA DI NOTIFICHE METEO

WeatherAlert

Relazione per il corso di Ingegneria del Software

Studente:
NYA EMMANUEL

Matricola:
7146768

Corso:
Ingegneria del Software

Docente:
Prof. Enrico Vicario

Anno Accademico 2025/2026

Indice

1 Analisi **2**
1.1 Contesto e statement . . . 2
1.2 Architettura e strumenti . . . 2

2 Progettazione **3**
2.1 Requisiti funzionali . . . 3
2.2 Casi d'uso . . . 3
2.3 Diagramma dei casi d'uso . . . 3
2.4 Use Case Templates . . . 4
2.4.1 UC1 — Aggiunta osservatore . . . 4
2.4.2 UC2 — Aggiornamento temperatura . . . 4
2.4.3 UC3 — Rimozione osservatore . . . 5
2.4.4 UC4 — Integrazione display legacy . . . 5
2.5 Diagramma delle classi . . . 6
2.6 Diagramma di sequenza . . . 7
2.7 Diagramma delle attività . . . 8

3 Implementazione **9**
3.1 Pattern Observer — `StazioneMeteo` . . . 9
3.2 Pattern Observer — osservatori concreti . . . 9
3.3 Pattern Adapter — `AdapterLegacy` . . . 10
3.4 Motivazione delle scelte progettuali . . . 10

4 Collaudo unitario **11**
4.1 Obiettivi dei test . . . 11
4.2 Test #1 — `StazioneMeteoTest` . . . 11
4.3 Test #2 — `AvvisoCaldoTest` . . . 12
4.4 Test #3 — `AdapterLegacyTest` . . . 12

1. Analisi

1.1 Contesto e statement

Il presente elaborato descrive la progettazione e l'implementazione di un sistema software in Java per il monitoraggio di condizioni meteo, applicando i design pattern **Observer** e **Adapter**.

Lo scenario prevede una stazione meteo centrale che rileva la temperatura. Diversi display devono aggiornarsi automaticamente al variare della temperatura. Uno dei display è un sistema *legacy* con un'interfaccia incompatibile: usa gradi Fahrenheit tramite un metodo con firma diversa, e non può essere modificato.

L'obiettivo è integrare tutti i display, compresi quelli incompatibili, in un sistema unificato che gestisca le notifiche in modo coerente.

- **Observer** — disaccoppia la stazione meteo dai display: ogni display si registra come osservatore e riceve automaticamente gli aggiornamenti.
- **Adapter** — adatta il display legacy (interfaccia Fahrenheit) all'interfaccia standard degli osservatori, senza modificarne il codice sorgente.

1.2 Architettura e strumenti

Il sistema è organizzato in tre package principali:

- `meteo.domain` — contiene `StazioneMeteo` (Subject del pattern Observer)
- `meteo.observer` — contiene l'interfaccia `Osservatore` e gli osservatori concreti (`DisplayTemperatura`, `AvvisoCaldo`)
- `meteo.adapter` — contiene `DisplayLegacy` (Adaptee) e `AdapterLegacy` (Adapter)

Strumenti utilizzati:

- Java 17 — linguaggio di implementazione
- JUnit 5 — framework per i test unitari
- PlantUML — per i diagrammi UML
- IntelliJ IDEA — ambiente di sviluppo

2. Progettazione

2.1 Requisiti funzionali

- **RF1** — *Registrazione osservatori:* il sistema deve permettere di aggiungere e rimuovere osservatori dalla stazione meteo.
- **RF2** — *Notifica automatica:* al variare della temperatura, tutti gli osservatori registrati devono essere notificati.
- **RF3** — *Display Celsius:* un osservatore deve mostrare la temperatura corrente in gradi Celsius.
- **RF4** — *Avviso caldo:* un osservatore deve emettere un avviso quando la temperatura supera una soglia configurabile.
- **RF5** — *Display legacy:* il sistema deve integrare un display con interfaccia incompatibile senza modificarne il codice.
- **RF6** — *Conversione unità:* l'adapter deve convertire correttamente la temperatura da Celsius a Fahrenheit con la formula $F = C \times \frac{9}{5} + 32$.
- **RF7** — *Collaudo unitario:* test JUnit su tutte le componenti principali.

2.2 Casi d'uso

Il sistema prevede un unico attore, l'**Utente**, che interagisce con la stazione meteo e osserva i risultati sui vari display. I casi d'uso principali sono:

- **UC1** — Aggiunta osservatore
- **UC2** — Aggiornamento temperatura (include UC4)
- **UC3** — Rimozione osservatore
- **UC4** — Integrazione display legacy

2.3 Diagramma dei casi d'uso

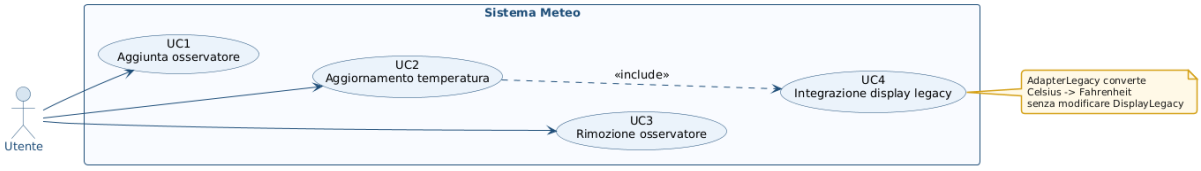

Figura 2.1: Diagramma dei casi d'uso

2.4 Use Case Templates

2.4.1 UC1 — Aggiunta osservatore

Caso d'uso 1 — Aggiunta osservatore	
Livello	User Goal
Attori	Utente
Pre-condizione	La `StazioneMeteo` è stata istanziata
Flusso base	1. L'utente crea un osservatore concreto (es. `DisplayTemperatura`). 2. L'utente chiama `stazione.aggiungi(osservatore)`. 3. Il sistema aggiunge l'osservatore alla lista interna.
Flusso alternativo	—
Post-condizione	L'osservatore è registrato e riceverà le notifiche successive.

2.4.2 UC2 — Aggiornamento temperatura

Caso d'uso 2 — Aggiornamento temperatura	
Livello	User Goal
Attori	Utente
Pre-condizione	Almeno un osservatore è registrato nella stazione
Flusso base	1. L'utente chiama `stazione.setTemperatura(valore)`. 2. Il sistema aggiorna il valore interno della temperatura. 3. Il sistema chiama `notifica()`, che itera sugli osservatori. 4. Ogni osservatore riceve il valore tramite `aggiorna()`. 5. Ogni osservatore reagisce secondo la propria logica.
Flusso alternativo	2a. Nessun osservatore registrato: il sistema non lancia eccezioni, la temperatura viene aggiornata senza notifiche.
Post-condizione	Tutti gli osservatori hanno ricevuto la nuova temperatura.

2.4.3 UC3 — Rimozione osservatore

Caso d'uso 3 — Rimozione osservatore	
Livello	User Goal
Attori	Utente
Pre-condizione	L'osservatore da rimuovere è registrato nella stazione
Flusso base	1. L'utente chiama `stazione.rimuovi(osservatore)`. 2. Il sistema rimuove l'osservatore dalla lista. 3. I successivi aggiornamenti non raggiungono l'osservatore rimosso.
Flusso alternativo	—
Post-condizione	L'osservatore non riceverà più notifiche.

2.4.4 UC4 — Integrazione display legacy

Caso d'uso 4 — Integrazione display legacy	
Livello	User Goal
Attori	Utente
Pre-condizione	Esiste un `DisplayLegacy` con interfaccia incompatibile
Flusso base	1. L'utente crea un `DisplayLegacy`. 2. L'utente crea un `AdapterLegacy` passando il `DisplayLegacy`. 3. L'utente registra l'`AdapterLegacy` nella stazione come `Osservatore`. 4. Al variare della temperatura, l'adapter riceve il valore in Celsius. 5. L'adapter converte in Fahrenheit e chiama `mostraFahrenheit()` su `DisplayLegacy`.
Flusso alternativo	—
Post-condizione	Il display legacy mostra la temperatura in Fahrenheit senza essere stato modificato.

2.5 Diagramma delle classi

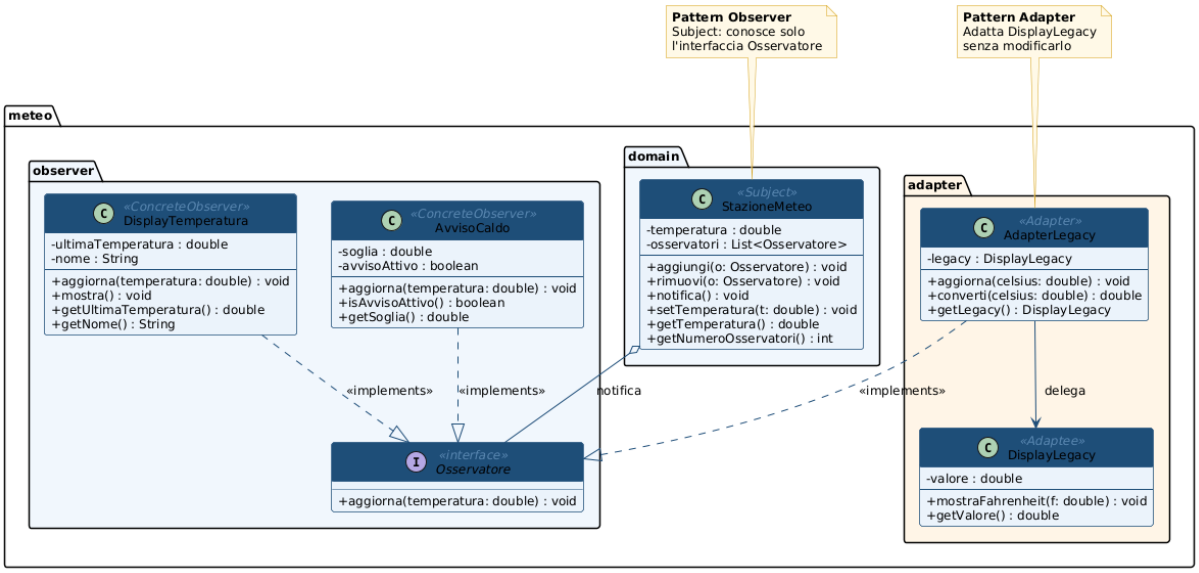

Figura 2.2: Diagramma delle classi

Il diagramma mostra le sei componenti principali: `StazioneMeteo` (Subject) usa l'interfaccia `Osservatore`; `DisplayTemperatura` e `AvvisoCaldo` la implementano; `AdapterLegacy` implementa `Osservatore` e contiene un riferimento a `DisplayLegacy` (Adaptee).

2.6 Diagramma di sequenza

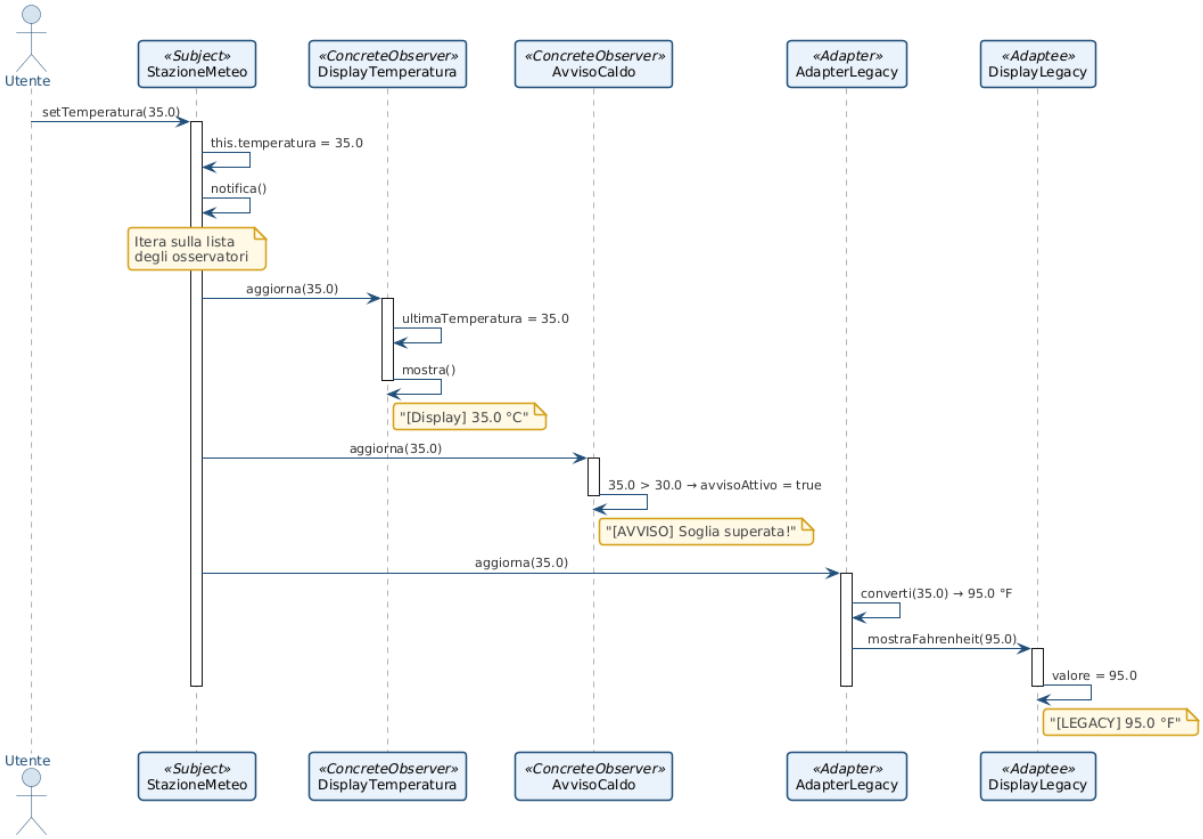

Figura 2.3: Diagramma di sequenza — `setTemperatura(35.0)`

La sequenza descrive l'interazione durante `setTemperatura(35.0)` con tre osservatori registrati. `StazioneMeteo` chiama `aggiorna(35.0)` su ciascuno: `DisplayTemperatura` stampa il valore in Celsius; `AvvisoCaldo` attiva l'avviso ($35 > 30$); `AdapterLegacy` converte a 95 °F e delega a `DisplayLegacy`.

2.7 Diagramma delle attività

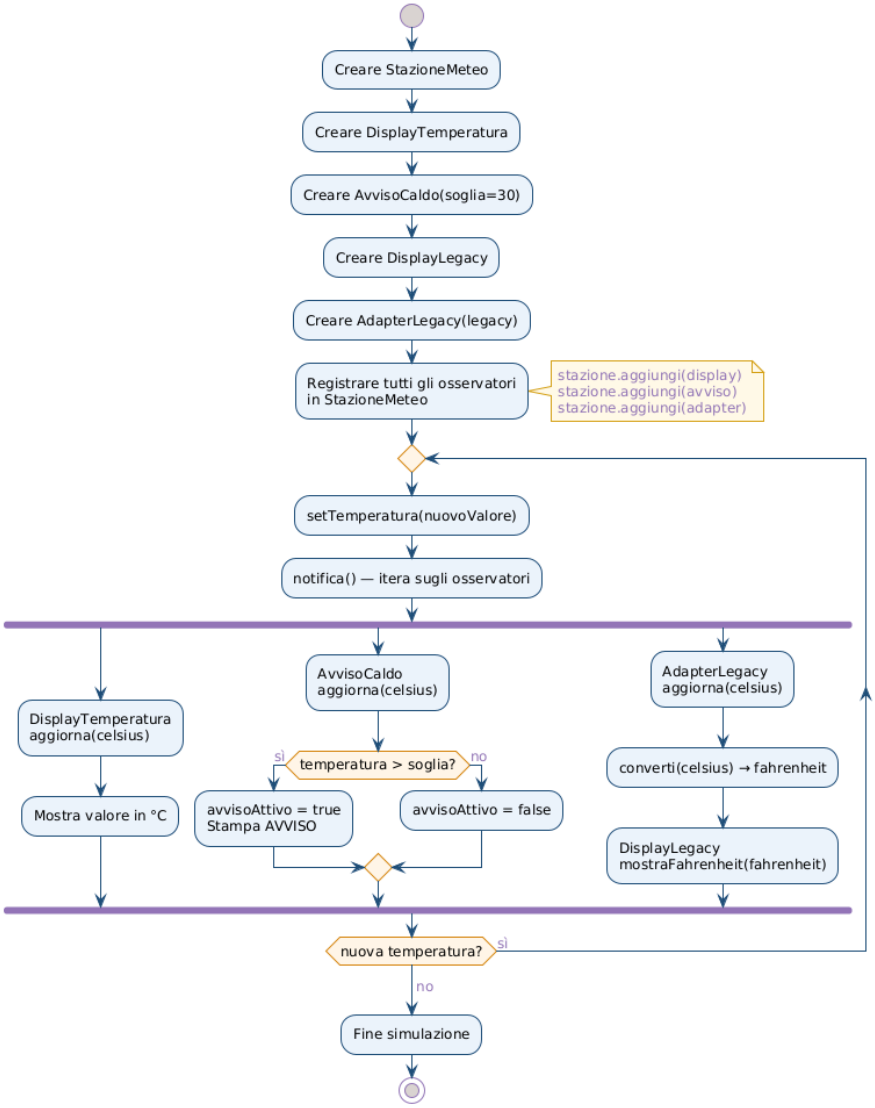

Figura 2.4: Diagramma delle attività — flusso principale

Il diagramma descrive il flusso dell'applicazione: dalla creazione degli oggetti, alla registrazione degli osservatori, fino alla notifica iterata ad ogni aggiornamento di temperatura.

3. Implementazione

3.1 Pattern Observer — `StazioneMeteo`

`StazioneMeteo` è il *Subject* del pattern Observer. Mantiene una lista di `Osservatore` e notifica tutti i registrati ogni volta che la temperatura viene modificata tramite `setTemperatura()`.

```java
public void setTemperatura(double temperatura) {
    this.temperatura = temperatura;
    notifica();
}

public void notifica() {
    for (Osservatore o : osservatori) {
        o.aggiorna(temperatura);
    }
}

public void aggiungi(Osservatore o) {
    osservatori.add(o);
}

public void rimuovi(Osservatore o) {
    osservatori.remove(o);
}
```

Listing 3.1: StazioneMeteo.java — notifica e aggiornamento

Ogni chiamata a `setTemperatura()` aggiorna il valore interno e poi itera sulla lista degli osservatori. `StazioneMeteo` conosce solo l'interfaccia `Osservatore`, rispettando il principio *Open/Closed*.

3.2 Pattern Observer — osservatori concreti

`DisplayTemperatura` implementa `Osservatore` e memorizza e stampa la temperatura corrente in Celsius.

```java
@Override
public void aggiorna(double temperatura) {
    this.ultimaTemperatura = temperatura;
    mostra();
}
```

Listing 3.2: DisplayTemperatura.java — aggiorna()

`AvvisoCaldo` implementa `Osservatore` con logica condizionale: se la temperatura supera la soglia configurata nel costruttore, attiva il flag `avvisoAttivo`.

```java
@Override
public void aggiorna(double temperatura) {
    if (temperatura > soglia) {
        this.avvisoAttivo = true;
        System.out.println("[AVVISO CALDO] Soglia superata!");
    } else {
        this.avvisoAttivo = false;
    }
}
```

Listing 3.3: AvvisoCaldo.java — logica della soglia

3.3 Pattern Adapter — `AdapterLegacy`

`AdapterLegacy` risolve l'incompatibilità tra l'interfaccia `Osservatore` (Celsius) e `DisplayLegacy` (Fahrenheit). Implementa `Osservatore` e contiene un riferimento al display legacy.

```java
@Override
public void aggiorna(double celsius) {
    double fahrenheit = converti(celsius);
    legacy.mostraFahrenheit(fahrenheit);
}

// Formula: F = C * 9/5 + 32
public double converti(double celsius) {
    return celsius * 9.0 / 5.0 + 32.0;
}
```

Listing 3.4: AdapterLegacy.java — conversione e delega

3.4 Motivazione delle scelte progettuali

Observer è stato scelto perchè consente di aggiungere e rimuovere osservatori a runtime senza modificare `StazioneMeteo`. Il Subject dipende solo dall'interfaccia `Osservatore`, rispettando il principio *Open/Closed*.

Adapter è stato scelto per integrare `DisplayLegacy` senza modificarne il codice sorgente. L'adapter funge da ponte tra le due interfacce incompatibili, mantenendo la piena compatibilità con il sistema Observer.

4. Collaudo unitario

4.1 Obiettivi dei test

I test JUnit 5 mirano a:

- Verificare la corretta registrazione e rimozione degli osservatori.
- Assicurare che le notifiche raggiungano tutti gli osservatori registrati.
- Verificare la logica condizionale di `AvvisoCaldo` (soglia).
- Verificare la correttezza della conversione Celsius/Fahrenheit in `AdapterLegacy`.
- Verificare che l'adapter si integri correttamente con `StazioneMeteo`.

4.2 Test #1 — `StazioneMeteoTest`

Verifica la registrazione degli osservatori e la propagazione delle notifiche. Tracciato a **UC1** e **UC2**.

```java
@Test
void testNotificaAggiornaDisplay() {
    stazione.aggiungi(display);
    stazione.setTemperatura(25.0);
    assertEquals(25.0, display.getUltimaTemperatura());
}

@Test
void testNotificaMultipliOsservatori() {
    stazione.aggiungi(display);
    stazione.aggiungi(avviso);
    stazione.setTemperatura(35.0);
    assertEquals(35.0, display.getUltimaTemperatura());
    assertTrue(avviso.isAvvisoAttivo()); // 35 > 30
}

@Test
void testOsservatoreNonRiceveDopoRimozione() {
    stazione.aggiungi(display);
    stazione.setTemperatura(20.0);
    stazione.rimuovi(display);
    stazione.setTemperatura(99.0);
    assertEquals(20.0, display.getUltimaTemperatura());
}
```

Listing 4.1: StazioneMeteoTest.java — test principali

4.3 Test #2 — `AvvisoCaldoTest`

Verifica la logica della soglia: l'avviso deve attivarsi solo con temperatura *strettamente maggiore* della soglia. Tracciato a **UC2**.

```java
@Test
void testAvvisoAttivoSopraLaSoglia() {
    avviso.aggiorna(35.0);
    assertTrue(avviso.isAvvisoAttivo());
}

@Test
void testAvvisoNonAttivoEsattamenteAllaSoglia() {
    avviso.aggiorna(30.0); // soglia = 30.0, condizione e' >
    assertFalse(avviso.isAvvisoAttivo());
}

@Test
void testAvvisoCambiaStatoCorrettamente() {
    avviso.aggiorna(35.0);
    assertTrue(avviso.isAvvisoAttivo());
    avviso.aggiorna(20.0);
    assertFalse(avviso.isAvvisoAttivo());
}
```

Listing 4.2: AvvisoCaldoTest.java — test sulla soglia

4.4 Test #3 — `AdapterLegacyTest`

Verifica la conversione Celsius/Fahrenheit e l'integrazione dell'adapter come `Osservatore`. Tracciato a **UC4**.

```java
@Test
void testConversioneZeroGradi() {
    assertEquals(32.0, adapter.converti(0.0), 0.001);
}

@Test
void testConversioneCentoGradi() {
    assertEquals(212.0, adapter.converti(100.0), 0.001);
}

@Test
void testConversioneTemperaturaNegativa() {
    // punto comune tra le due scale
    assertEquals(-40.0, adapter.converti(-40.0), 0.001);
}

@Test
void testAdapterComportaComeOsservatore() {
    StazioneMeteo stazione = new StazioneMeteo();
    stazione.aggiungi(adapter);
    stazione.setTemperatura(0.0);
```

```
    assertEquals(32.0, legacy.getValore(), 0.001);
}
```

Listing 4.3: AdapterLegacyTest.java — conversione e integrazione